In cpp recursion program of factorial if we write n-- or --n instead of n-1 in the recursive call

Good question 👍 This is a very important concept in C++ recursion.

Let's understand clearly and simply.

🔹 Normal Factorial Recursion

Correct version:

```
    if (n == 0)
        return 1;
    return n * fact(n - 1);
}
```

Here:

- We send (n - 1) to next recursive call.
- Original `n` does NOT change.
- Each call gets correct value.

Example for `fact(5)` :

C++

```
4 * fact(3)
3 * fact(2)
2 * fact(1)
1 * fact(0)
```

Works perfectly ✅

🔴 What if we use `n--` ?

Code

C++

Meaning:

- First, it sends current value of `n`
- Then decreases `n`

So if `n = 5` :

- `fact(n--)` sends 5
- Then `n` becomes 4
- But recursive call receives 5 again 😱

So it becomes:

```
→ fact(5)
→ fact(5)
→ fact(5)
```

It never reaches 0 ❌

This causes infinite recursion → stack overflow

🔴

Code

```
return n * fact(--n);
```

C++

Meaning:

- First decrease `n`
- Then send new value

If `n = 5`:

- `--n` makes it 4
- Then `fact(4)` is called

Sounds okay? 🤔

But problem is:

We are also using `n` in multiplication.

So expression:

```
return n * fact(--n);
```

C++

This causes undefined behavior.

The compiler may:

- Multiply first
- Decrement first
- Or behave unpredictably

So result is unreliable ❌

✅ Why `n - 1` is Correct

```
fact(n - 1)
```

C++

- Clean and safe
- No undefined behavior

🔥 Important Rule in C++

❌ Never modify a variable and use it in same expression like:

```
n * fact(--n)
```

</> C++

📌 Final Summary

Expression	What Happens	Safe?
`fact(n - 1)`	Sends n-1	✅ Correct
`fact(n--)`	Sends n (same value)	❌ Infinite recursion
`fact(--n)`	Modifies and uses n together	❌ Undefined behavior

🎯 Best Practice

Always write:

```
return n * fact(n - 1);
```

</> C++

If you want, I can show step-by-step memory stack visualization also 🔍

Explain in detail both cases of decrement step by step dry run it step by step in details

2/2

Very good 👌 Let's do complete dry run step-by-step for both:

1. `fact(n--)`
2. `fact(--n)`

Assume this factorial function:

```
    if (n == 0)
        return 1;
    return n * fact( ??? );
}
```

We will replace `???` with:

- `n--`
- `--n`

🔴 CASE 1: Using `n--` (Post-Decrement)

C++

```
return n * fact(n--);
```

C++

- First use current value
- Then decrease it

🔎 Dry Run: fact(3)

🔹 First Call

```
n = 3
```

Expression:

```
return n * fact(n--)
```

```
n * fact(3)
```

- `n` (for multiplication) = 3
- `n--` sends 3 to recursive call
- THEN `n` becomes 2 (inside current function only)

So it becomes:

```
3 * fact(3)
```

```
Re
```

🔹 Second Call

```
n = 3
```

Same process:

```
3 * fact(3)
```

Code

Again:

```
3 * fact(3)
```

Code

❌ What happens?

It never becomes:

```
fact(1)
fact(0)
```

So base case is NEVER reached.

Program keeps calling:

Code

```
fact(3)
fact(3)
...
```

Eventually:

💥 Stack Overflow

💥 Program crashes

Code

🧠 Why This Happens?

Because:

`n--` sends original value before decrement.

So recursive call never receives smaller number.

🔴 CASE 2: Using `--n` (Pre-Decrement)

```
return n * fact(--n);
```

‹/› C++

⚠ Important Rule of `--n`

- First decrease value
- Then use it

BUT ❗

In C++, order of evaluation inside expression like:

```
n * fact(--n)
```

‹/› Code

That means compiler may:

- Evaluate `n` first
- Or evaluate `--n` first

This causes undefined behavior.

🔎 Let's Assume One Possible Order

We assume compiler evaluates left to right (just for understanding).

Dry Run: fact(3)

- First Call

```
n = 3
```

Expression:

```
return n * fact(--n);
```

```
Ex
```

- `--n` makes n = 2
- Recursive call becomes `fact(2)`

Now multiplication uses `n`

But what is `n` now?

It is 2 (already decreased)

So expression becomes:

```
2 * fact(2)
```

Already wrong.

🔹 Second Call

```
n = 2
```

Same thing:

Code

Expression becomes:

```
1 * fact(1)
```

Code

- Third Call

```
n = 1
```

--n → n = 0

Code

```
0 * fact(0)
```

Code

- Fourth Call

```
return 1
```

Now backtracking:

Code

```
1 * 0 = 0
2 * 0 = 0
```

Final Answer = 0 ❌

Factorial of 3 should be 6

Code

⚠ Why This is Worse?

Because:

We modified `n` and used it in same expression.

C++ says:

👉 Modifying and reading same variable in one expression without sequencing = Undefined Behavior

Compiler may:

- Multiply first
- Decrement first
- Do something else

So result is unpredictable.

🟢 Compare With Correct Version

```
return n * fact(n - 1);
```

C++

```
2 * fact(1)
1 * fact(0)
```

Backtracking:

Code

```
2 * 1 = 2
3 * 2 = 6
```

Correct ✅

Code

Expression	What Happens	Result
`fact(n--)`	Sends same value again	Infinite recursion
`fact(--n)`	Changes n before multiply	Wrong / Undefined
`fact(n-1)`	Safe copy of value	Correct

🎯 Golden Rule

In recursion:

✔ Never change original variable inside recursive call

✔ Always pass modified copy like `n-1`

If you want, I can also draw stack memory diagram visualization for even deeper understanding 👀

return n * fact(--n);

When we write this and call the function it receive n is 3 and left to right calculate 3*fact(2) will call .... Same for others as n become 1 it return 1 and back tracking start

Very good 👌 You are thinking correctly — but there is a very subtle C++ rule here that changes everything.

You said:

> Left to right calculate 3 * fact(2)

That looks logical...
But in C++, this assumption is not guaranteed.

Let's break this VERY carefully.

🔴 Expression

```
return n * fact(--n);
```

C++

```
fact(3);
```

C++

```
n = 3
```

Code

⚠ VERY IMPORTANT RULE (C++)

In the expression:

```
return fact(--n)
```

C++

- Left side (`n`) is evaluated first

  OR
- Right side (`fact(--n)`) is evaluated first

The order is unspecified.

And this is where the danger begins.

🔵 Case A: Suppose compiler evaluates LEFT SIDE first

Step I:

Evaluate `n`

```
n = 3
```

Code

Ev

`--n` makes:

```
n = 2
```

Code

```
fact(2)
```

Code

```
3 * fact(2)
```

Code

Now inside fact(2)

```
return n * fact(--n);
```

Same assumption (left first):

Code

```
n = 2
```

Code

```
fact(1)
```

Expression:

Code

```
2 * fact(1)
```

Code

Next level fact(1)

```
n = 1
```

Code

```
n = 1
```

Code

```
fact(0)
```

Expression:

Code

```
fact(0)
```

Code

fact(0)

Returns:

```
1
```

Code

Backtracking

```
2 * 1 = 2
3 * 2 = 6
```

✅ Final answer = 6

Code

Step I:

Evaluate `fact(--n)`

```
--n → n = 2
```

Code

```
fact(2)
```

Code

But what is `n` now?

It is already 2, not 3.

So expression becomes:

```
2 * fact(2)
```

Code

Continue inside fact(2)

Again assume right side first.

```
fact(1)
```

Then multiply by current n (I):

Code

```
1 * fact(1)
```

Code

fact(1)

```
fact(0)
```

Multiply by 0:

Code

```
0 * fact(0)
```

Code

Backtracking

```
0 * 1 = 0
1 * 0 = 0
2 * 0 = 0
```

❌ Final answer = 0

🚨 Why This Is Undefined Behavior

Code

```
n * fact(--n)
```

C++

- Modifying `n`
- Without sequence guarantee

C++ standard says this is undefined behavior.

That means:

- Sometimes result 6
- Sometimes result 0
- Sometimes something else
- Depends on compiler and optimization

🔥 Key Concept

When you write:

```
return n * fact(--n);
```

C++

This is NOT safe in C++.

🟢 Why `n - 1` Is Safe

```
return n * fact(n - 1);
```

C++

- `n` is not modified
- `n - 1` is just a temporary value
- No undefined behavior
- Fully predictable

🎯 Final Clear Answer To Your Thought

You said:

> Left to right calculate 3 * fact(2)

That is only TRUE if compiler evaluates left side first.

But C++ does NOT promise that.

That's why this code is dangerous.

If you want, I can now explain:

- What "sequence points" mean
- Or how C++17 changed some evaluation rules
- Or show memory stack diagram visually

Tell me which level you want 👌

> Is there any other language which promise for correct result for pre decrement case

Excellent question 👌 Now you are thinking like a language designer.

Short answer:

✅ Yes, some languages define strict evaluation order,
❌ but almost no language recommends writing code like `n * fact(--n)`, because it's logically confusing.

Let's go step by step.

🔴 Why C++ Fails Here

In C++:

```
return n * fact(--n);
```

C++

C++ prioritizes performance over strict evaluation order.

🟢 Languages That Guarantee Left-to-Right Evaluation

Some languages define strict evaluation order for expressions.

🟢 I. Java

In Java

Java guarantees:

> Operands are evaluated strictly left-to-right.

So:

```
return n * fact(--n);
```

Java

1. Read `n`
2. Then evaluate `fact(--n)`

So for `n = 3`:

- Left side → 3
- `--n` → 2
- Call `fact(2)`

This will consistently produce:

```
3 2 1 – 0
```

Code

BUT ⚠ still considered bad style

🟢 2. C#

In C#

C# also guarantees left-to-right evaluation.

So:

```
return n * Fact(--n);
```

C#

🟢 3. Python

In Python

Python evaluates left-to-right.

But:

❌ Python does NOT even have `--n` operator.

You would write:

```
return n * fact(n-1)
```

Python

Very safe design.

🟡 Important Clarification

Even in Java or C#:

Although result is defined, the logic is still dangerous because:

You are modifying parameter `n` inside expression.

It makes code:

- Hard to read
- Easy to misunderstand
- Error-prone in complex expressions

Professional developers NEVER write factorial like this.

🔥 Why Most Modern Languages Avoid This Problem

Modern languages prefer:

- Clear sequencing
- No side effects in expressions
- Functional style

Because expressions like:

```
n * fact(--n)
```

C++

- Mutation
- Recursion

- Expression evaluation

Which is conceptually messy.

🎯 Final Answer

Yes — languages like:

- Java
- C#
- Python (different syntax but defined order)

guarantee evaluation order.

But:

✔ Only Java and C# would give consistent result for `n * fact(--n)`
✔ C++ does NOT
✔ Good programmers never write it this way in any language

If you want, I can now explain:

- Why C++ was designed this way (performance reason)
- Or how compilers internally evaluate such expressions
- Or what "sequence points" are in deep detail

You are asking very advanced questions now 👏